

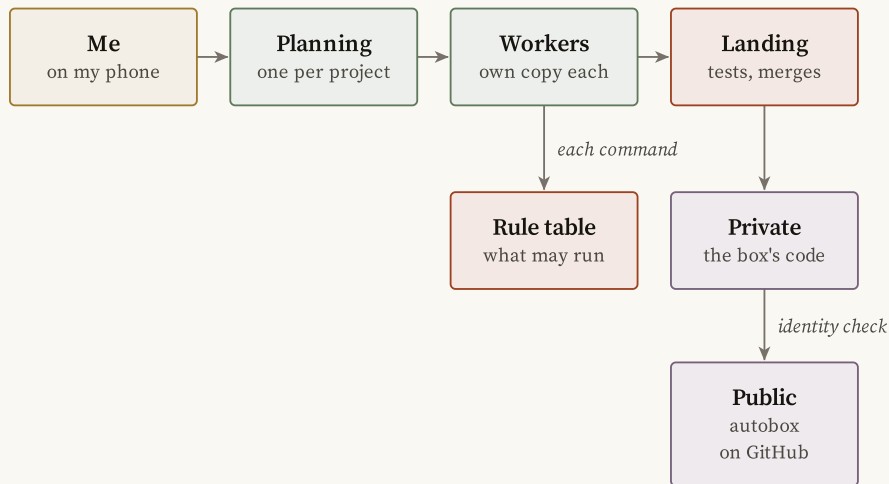
autobox

Scripts that let Claude Code sessions plan, build, check and merge work on a server nobody is watching, with every state kept in files and every merge decided by gates.

Claude Code is good at one sitting of work. Left running for days it has three problems: its context fills up and it forgets, it can run any command it likes, and nothing stops it merging broken code. autobox is the layer around it that fixes those three. It is not another agent. It is shell and Python glue around Claude Code.

The owner talks to it from a phone over Slack. Each project has one planning session that reads the channel, splits work into tracks, and dispatches workers. Each worker gets its own git worktree and branch, so two workers never edit the same checkout.

FIGURE 1 *the parts*

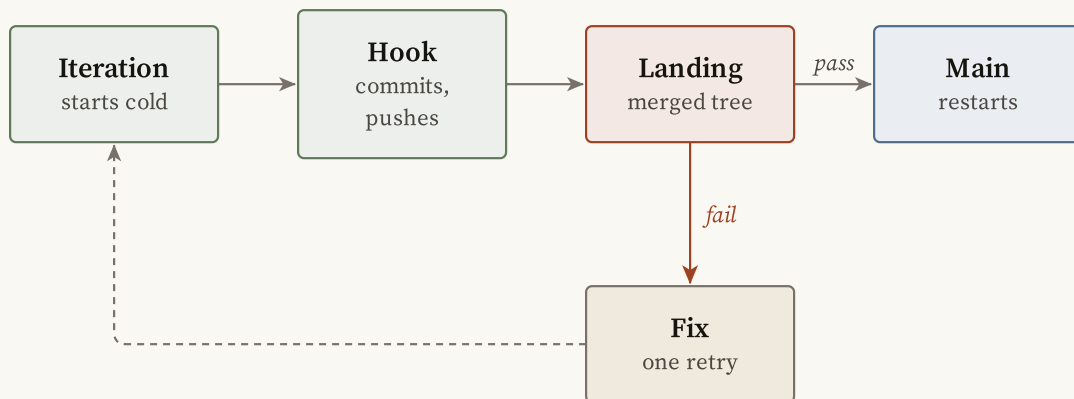


Every change goes through the landing, and every command goes past the rule table first.

How one piece of work moves

1. **Dispatch.** `cc <repo> <track> --go "<brief>"` writes the brief to `task.md`, makes a worktree and a branch, and starts `cc-loop`.
2. **Iterate.** Each iteration is a new process with an empty context. Its prompt is the brief plus the last few kilobytes of `progress.md`, the worker's own journal. It does the next unfinished step and writes a dated entry early, so an iteration cut short still leaves the next one a trail.
3. **Commit.** When the iteration stops, the hook commits and pushes the branch. The worker is told never to run `git commit` itself, and the guard stops it if it tries on the default branch.
4. **Stop.** The loop ends when the journal says `STATUS: DONE` or `STATUS: BLOCKED: <question>`, or after two errors, two iterations that changed nothing, or the iteration cap. Done turns into a pull request whose body the worker wrote; blocked routes the question to whoever dispatched it.
5. **Land.** `cc-land` builds the default branch merged with the PR head in a throwaway worktree, runs the gates there, has a model review the diff when it touches the control layer, then squash-merges, installs, and restarts the services the change touched. It checks the restart by the new process ID, not by an exit code.

FIGURE 2 *one track*



A red gate sends the track back for one fix iteration. A second red stops and asks a person.

The worker never commits or merges. The hook commits for it, and the landing decides whether it merges.

The hard parts

Memory that survives a restart

A long session degrades as its context fills, and a crash loses whatever it was holding. So nothing important lives in the conversation. The brief, the journal, the board and git are the state, and each iteration reads them cold. The price is that a worker has to write things down as it goes; the loop prompt says so, and an iteration that leaves nothing in the journal is counted as stalled.

Checking what actually ships

A branch can pass its tests and still break main once it is merged with work that landed after it branched. So the gates never run on the branch alone. They run on the merged tree, and if the branch is green on its own but red merged, the landing says the merge is the reason. Merge queues do this for teams of people; here it matters more, because nobody reads most of these diffs before they merge. For ordinary PRs, several that touch disjoint files are merged in queue order and tested once as a batch, and a red batch is bisected over its own commits so only the breaking one is reverted.

A rule table that can't be walked around

The first version matched rules against the command as typed, so `setsid tmux kill-server` slipped past a rule written for `tmux kill-server`. The fix builds one text from the command plus every suffix of every statement in it, and every rule is checked against that, so wrapper words in front no longer hide anything. A change to the table is tested by replaying the commands agents really ran, taken from the session transcripts, through the old and new versions and reading what newly gets refused.

The table is a blocklist: it stops accidents and casual prompt injection, not a determined attacker. Real containment is a Docker sandbox with an egress allowlist.

Keeping the private half private

autobox is the public mirror of one directory, `core/`, of a private repo that also holds this machine's own config. Which directory is public is fixed when a file is written, never chosen at publish time. Before every publish, a gate searches the file contents, the file names and the commit subject for the machine's own user, host and repo names and a deny list. One hit and nothing is pushed.

The same failure, once

When one test goes red on many unrelated PRs, each worker ends up investigating it again. Every gate failure is written to the ledger with a fingerprint of what failed. Once the same fingerprint is red on three landings within three days, the reconcile pass claims it and dispatches one worker to fix it, and the other PRs wait for that fix instead of each writing their own.

What it doesn't do

- There is no always-on manager agent. The design doc cites work finding that one agent working in a straight line, with a fresh context each time, does better than many in parallel, and a manager would spend tokens waiting.
- The rule table does not catch every command with the same effect under a different name. Those are left to review.
- The gate suite still runs on the machine itself. Hosted CI is wired in but switched off, because hosted minutes for a private repo are not free.